Ex.No. 1 Git Repository initialization and its operations

Aim

To initialize a Git repository and perform basic version control operations such as creating a repository, adding files, committing changes, and checking the status and history of the repository using Git commands.

Procedure

GitHub Account Creation

- Open a web browser and go to the GitHub website: https://github.com
- Click on the **Sign up** button.
- Enter the required details such as username, email address, and password.
- Verify the email address by entering the code sent to the registered email.
- Complete the account setup by answering the required questions (usage purpose, experience level, etc.).
- Choose a free plan (or other plan if required) and finish the registration process.
- Log in to the GitHub account using the created username and password.

Repository Creation on GitHub

- After logging in, click on the **"+"** icon at the top-right corner of the GitHub homepage.
- Select **New repository** from the dropdown menu.
- Enter a **Repository name**.
- Provide a short **description** (optional).
- Choose the repository visibility as **Public** or **Private**.
- (Optional) Select **Initialize this repository with a README** file.
- (Optional) Add a .gitignore file and choose a license if required.
- Click on the **Create repository** button.

- The new repository is now successfully created and ready for use.

Basic operations on Git

- Open the terminal or command prompt on the system.
- Navigate to the required project directory using the **cd** command.
- Initialize a new Git repository by executing the command **git init**.
- Create a new file or modify an existing file in the project directory.
- Check the status of the repository using the command **git status**.
- Add the file(s) to the staging area using the command **git add <filename> or git add .** to add all files.
- Verify that the files are staged by again using the **git status** command.
- Commit the staged files to the repository with an appropriate message using the command **git commit -m "commit message**".
- View the commit history using the command **git log**.
- Make further changes to files and repeat the add and commit steps to track updates in the repository.

Result:

A Git repository was successfully initialized, and basic version control operations such as adding files, committing changes, and viewing the repository status and commit history were performed successfully using Git commands.

Ex.No 2 Link a local repository with GitHub and sync code.

Aim

To link a local Git repository with a remote GitHub repository and synchronize the code by pushing local commits to GitHub and pulling updates from the remote repository using Git commands.

Procedure

1. Ensure that Git is installed on the system and a local Git repository is already initialized.
2. Log in to the GitHub account and create a new repository (without or with a README as required).
3. Copy the remote repository URL (HTTPS or SSH) from GitHub.
4. Open the terminal or command prompt and navigate to the local repository directory.
5. Add the remote GitHub repository using the command:
   git remote add origin <repository-URL>
6. Verify the remote repository connection using the command:
   git remote -v
7. Stage the required files using the command:
   git add .
8. Commit the changes with an appropriate message using the command:
   git commit -m "Initial commit"
9. Push the local commits to the GitHub repository using the command:
   git push -u origin main (or master, depending on the branch name).
10. To synchronize updates from GitHub to the local repository, use the command:
    git pull origin main

11. Confirm that the local and remote repositories are successfully synchronized.

Result:

The local Git repository was successfully linked to the remote GitHub repository, and the code was synchronized by pushing local commits to GitHub and pulling updates from the remote repository using Git commands.

Ex.No:3 Create and merge branches for feature development

Aim:

To create multiple branches in a Git repository for feature development and successfully merge the feature branch into the main branch using Git commands.

Procedure

- Open the terminal or command prompt and navigate to the existing Git repository.
- Check the current branch and existing branches using the command: **git branch**
- Create a new branch for feature development using the command: **git branch feature1**
- Switch to the newly created branch using the command: **git checkout feature1***(Alternatively, use git checkout -b feature1 to create and switch in one step.)*
- Make the required changes to the project files for the feature.
- Stage the modified files using the command: **git add .**
- Commit the changes with a suitable message using the command: **git commit -m "Implemented feature1"**
- Repeat steps 3 to 7 to create additional feature branches (e.g., feature2, feature3) as needed.
- After completing feature development, switch back to the main branch using:
  git checkout main
- Merge a feature branch into the main branch using the command: **git merge feature1**

- Resolve any merge conflicts if they occur, add the resolved files, and commit the merge.
- Verify the successful merge by checking the branch list and commit history using:
  git branch and git log.

Result

Multiple branches were successfully created in the Git repository for feature development, and the feature branch(es) were merged into the main branch without errors, confirming successful branch management and integration using Git commands.

Ex.No:4 Collaborate via forks and pull requests

Aim

To collaborate on a GitHub project by creating a fork of a repository, making changes in the forked repository, and submitting those changes to the original repository through pull requests.

Procedure

- Log in to the GitHub account and navigate to the original repository to be collaborated on.
- Click the **Fork** button at the top-right corner of the repository page to create a copy of the repository in your GitHub account.
- After the fork is created, copy the URL of the forked repository.
- Open the terminal or command prompt and clone the forked repository to the local system using:**git clone <forked-repository-URL>**
- Navigate into the cloned repository directory using **cd <repository-name>.**
- (Optional but recommended) Add the original repository as an upstream remote using:**git remote add upstream <original-repository-URL>**
- Create a new branch for making changes using: **git checkout -b feature-branch**
- Make the required changes to the project files.
- Stage the changes using:**git add .**
- Commit the changes with a clear message using: **git commit -m "Description of changes"**
- Push the changes to the forked repository on GitHub using: **git push origin feature-branch**
- Open the forked repository on GitHub and click **Compare & Pull Request**.

- Add a title and description explaining the changes made.
- Click **Create Pull Request** to submit the changes to the original repository.
- Wait for the repository maintainer to review, approve, and merge the pull request.

Result

The repository was successfully forked, changes were made and committed in the forked repository, and a pull request was created and submitted to the original repository, enabling effective collaboration and code review on GitHub.

Ex.No. 5 Create a basic GitHub Actions workflow

Aim

To create a basic GitHub Actions workflow that automates tasks such as building, testing, or deploying a project whenever changes are pushed to the repository, thereby enabling continuous integration and improving development efficiency.

Procedure

1. Create a GitHub Repository
   - Log in to your GitHub account.
   - Click on New Repository.
   - Provide a repository name and click Create Repository.
2. Create a Workflow Directory
   - Inside your repository, create a folder structure:
     - .github/workflows
   - This is the default directory where GitHub looks for workflow files.
3. Create a Workflow File
   - Inside the workflows folder, create a new YAML file (for example): main.yml
4. Define the Workflow Configuration
   - Open main.yml and add the following sample configuration:

```
name: Basic CI Workflow
on:
  push:
    branches: [ "main" ]
  pull_request:
```

```
    branches: [ "main" ]
  jobs:
   build:
    runs-on: ubuntu-latest
    steps:
     - name: Checkout repository
       uses: actions/checkout@v3
     - name: Display a message
       run: echo "Hello, GitHub Actions!"
```

5. Commit and Push the Workflow File
    - Save the file.
    - Commit the changes.
    - Push the file to the main branch.
6. Verify Workflow Execution
    - Go to your repository on GitHub.
    - Click on the Actions tab.
    - You will see the workflow running automatically after the push.
    - Click on the workflow run to view logs and results.
7. Modify as Needed
    - You can add steps such as installing dependencies, running tests, or deploying the application.
    - Commit and push changes to update the workflow.

Result:

A basic GitHub Actions workflow is successfully created and automatically runs when code is pushed to the repository.

Ex.No 6 Automate test runs using GitHub Actions.

Aim

To automate the execution of project test cases using GitHub Actions, so that tests are automatically triggered on code pushes or pull requests, ensuring continuous integration, early detection of errors, and improved software quality.

Procedure

1. **Create or Open a GitHub Repository**

    - Log in to GitHub.
    - Create a new repository or open an existing project repository.

2. **Organize Project with Test Files**

    - Ensure your project contains proper test files (e.g., test_*.py, *.test.js, etc.).
    - Verify tests run successfully on your local machine before automating.

3. **Create Workflow Directory**

    - Inside your repository, create the following folder structure:

      .github/workflows

4. Create a Workflow YAML File

    - Inside the workflows folder, create a file such as:

      test.yml

5. Define the Workflow Configuration

- Add the following sample configuration (example for a Python project):

```
name: Run Tests
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.x'
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
      - name: Run tests
```

```
run: |
  pytest
```

6. **Commit and Push the Workflow File**
    - Save the file.
    - Commit the changes.
    - Push to the main branch.
7. **Verify Test Execution**
    - Go to the repository on GitHub.
    - Click the **Actions** tab.
    - View the workflow run and check logs for test results.
8. **Monitor and Improve**
    - If tests fail, fix the errors locally and push changes.
    - Extend the workflow by adding code coverage, multiple OS testing, or different language versions if required.

Result:

Test cases are automatically executed whenever code is pushed or a pull request is created, ensuring continuous testing and improved code reliability.

Ex.No. 7 Use secrets in GitHub Actions securely

Aim

To securely store and use sensitive information such as API keys, passwords, and tokens in GitHub Actions by using encrypted Secrets, ensuring that confidential data is protected during automated workflows and not exposed in code repositories or logs.

Procedure

1. Create a GitHub Repository

   Create or open a repository in GitHub where you want to run automated workflows using GitHub Actions.
2. Navigate to Repository Settings

   Go to the repository → Click Settings → Select Secrets and variables → Click Actions
3. Add a New Secret

   Click New repository secret and enter:

   - Name (e.g., API_KEY, DB_PASSWORD)
   - Value (the sensitive information such as tokens or passwords).
     Then click Add secret.

4. Create or Edit Workflow File
   Inside the repository, create or open the workflow file located in the .github/workflows/ folder (for example main.yml).
5. Reference the Secret in Workflow
   Use the secret in the workflow file using the syntax:
   ${{ secrets.SECRET_NAME }}
   Example:
   ```
   env:
    API_KEY: ${{ secrets.API_KEY }}
   ```

6. **Use the Secret in Workflow Steps**
   Access the secret inside workflow steps such as scripts or commands.
   Example:

```
- name: Use API Key
run: echo "Using secret safely"
env:
API_KEY: ${{ secrets.API_KEY }}
```

7. **Run the Workflow**
Commit and push the workflow file. The workflow will run automatically and use the stored secret securely without exposing it in the repository.

8. **Follow Security Best Practices**

- Do not print secrets in logs.
- Use repository or organization secrets instead of hard-coding credentials.
- Rotate secrets periodically for better security.

Result:

The sensitive information such as API keys, passwords, and tokens was successfully stored and accessed securely using **Secrets** in GitHub Actions within the GitHub repository.

Ex.No 8 Deploy a static site via CI/CD pipeline

Aim

To design and implement a CI/CD pipeline using GitHub Actions in GitHub to automatically build and deploy a static website, ensuring continuous integration, faster updates, and reliable deployment whenever changes are pushed to the repository.

Procedure

1. Create a Repository

   Create a repository in GitHub and upload the static website files (HTML, CSS, JavaScript).

2. Organize Project Files

   Ensure the static website files such as index.html, style.css, and scripts are stored in the repository.

3. Enable CI/CD Workflow

   - Use GitHub Actions to automate the build and deployment process.

4. Create Workflow Directory

   - Inside your repository, create the following folder structure:

     .github/workflows

5. Create a Workflow YAML File

   - Inside the workflows folder, create a file such as:

     deploy.yml

6. Configure the Workflow

   Define the workflow to trigger when code is pushed to the repository. Example structure:

```
name: Deploy Static Site

on:

  push:

    branches: [main]



jobs:

  build-and-deploy:

    runs-on: ubuntu-latest

    steps:

    - name: Checkout code

      uses: actions/checkout@v3
```

7. Add Deployment Step
   Configure deployment to a hosting service such as GitHub Pages.
8. Commit and Push the Workflow File
   Save the workflow file and push the changes to the repository.
9. Automatic Deployment
   When new changes are pushed, the CI/CD pipeline automatically builds and deploys the static site.

10. Access the Deployed Website

    After successful execution, the static website will be available through the hosting platform URL.

Result:

The static website was successfully built and deployed using a CI/CD pipeline configured with GitHub Actions in GitHub.

Ex.No. 9 Use secrets in GitHub Actions securely

Aim

To automate the process of building a Docker container image using a CI workflow in GitHub Actions within a GitHub repository, ensuring consistent, reliable, and automated container image creation for application deployment using Docker.

Procedure

1. **Create a Repository**
Create or open a project repository in **GitHub** where the application source code will be stored.

2. **Add a Dockerfile**
In the root directory of the project, create a Dockerfile that contains instructions to build the container image using **Docker**.

3. **Push the Project to GitHub**
Upload the application code along with the Dockerfile to the GitHub repository.

4. **Create Workflow Directory**
Inside the repository, create a folder named .github/workflows/.

5. **Create a Workflow File**
Add a workflow file such as docker-build.yml in the workflows folder to define the automation process.

6. **Configure the Workflow Trigger**
Set the workflow to run when code is pushed to the main branch.

Example:

```
name: Build Docker Image
on:
  push:
    branches: [main]
```

7. **Add Job to Build Docker Image**
Configure the workflow to run on a virtual machine and check out the repository code using **GitHub Actions**.

Example steps:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout repository
      uses: actions/checkout@v3
```

8. **Build the Docker Image**
Add a step to build the Docker image.

Example:

```
    - name: Build Docker Image
      run: docker build -t my-app-image .
```

9. **Commit and Push the Workflow File**
Save the workflow file and push it to the repository.

10. **Automatic Execution**
Whenever new code is pushed, **GitHub Actions** automatically runs the workflow and builds the Docker image.

Result:

The Docker image for the application was successfully built using an automated workflow configured in GitHub Actions within the GitHub repository.

Ex.No. 10 Use secrets in GitHub Actions securely

Aim:

To configure an automated build process by creating a Jenkins job that is triggered through a webhook from GitHub, enabling Jenkins to automatically start the build whenever code changes are pushed to the repository, thereby supporting continuous integration and faster development workflows.

Procedure:

1. Install and Start Jenkins
Install and run Jenkins on your system or server.

2. Create a New Jenkins Job

- Open the Jenkins dashboard.
- Click New Item.
- Enter a job name and select Freestyle Project.
- Click OK.

3. Configure Source Code Management
In the job configuration page:

- Select Git under Source Code Management.
- Enter the repository URL from GitHub.

4. Enable GitHub Webhook Trigger
In the Build Triggers section, select GitHub hook trigger for GITScm polling.

5. Add Build Steps
In the Build section, add steps such as Execute shell and provide the commands required to build or test the project.

6. Save the Jenkins Job
Click Save to store the configuration.

7. Configure Webhook in GitHub

- Open the GitHub repository.
- Go to Settings → Webhooks.
- Click Add webhook.

- Enter the Payload URL (e.g., http://your-jenkins-url/github-webhook/).
- Select application/json as the content type.
- Choose Just the push event.

8. Test the Integration
Push changes to the GitHub repository. The webhook will notify Jenkins and automatically trigger the configured job.

9. Verify the Build Execution
Check the Build History in Jenkins to confirm that the job was triggered successfully after the GitHub push event.

Result:

The Jenkins job was successfully configured and triggered automatically through a webhook from GitHub.

Ex.No.11 Analyze and fix failing CI workflows.

Aim:

To identify, analyze, and resolve errors in failing CI workflows in GitHub Actions within a GitHub repository, ensuring successful automated builds, improved workflow reliability, and efficient continuous integration processes.

Procedure:

1. Access the Repository
Open the project repository in GitHub where the CI workflow is configured.

2. Navigate to Workflow Runs
Go to the Actions tab to view all workflow executions in GitHub Actions.

3. Identify the Failed Workflow
Look for workflow runs marked with a failed status (✕) and select the specific run to view details.

4. Review Error Logs
Open the failed job and check the log output of each step to identify the exact error message or failure point.

5. Analyze the Cause of Failure
Determine the reason for failure such as:

- Incorrect commands or scripts
- Dependency installation errors
- Configuration mistakes in the workflow file
- Missing environment variables or secrets

6. Modify the Workflow or Code
Edit the workflow file (.github/workflows/*.yml) or project source code to fix the identified issue.

7. Commit and Push the Changes
Save the corrections and push the updated files to the repository.

8. Re-run the Workflow
Trigger the workflow again automatically through a new commit or manually select Re-run jobs in GitHub Actions.

9. Verify Successful Execution
Check the workflow status to confirm that the pipeline completes successfully without errors.

Result

The failing CI workflow was successfully analyzed by reviewing the logs and identifying the source of the error in GitHub Actions within the GitHub repository.

Ex.No.12 Design and implement a full CI/CD pipeline

Aim:

To design and implement a complete CI/CD pipeline using GitHub Actions in a GitHub repository to automate the processes of code integration, testing, building, and deployment, thereby improving software quality, development efficiency, and continuous delivery of applications.

Procedure:

1. Create a Source Code Repository
Create a project repository in GitHub and upload the application source code.

2. Plan the CI/CD Stages
Define the stages of the pipeline such as:

- Code integration
- Build process
- Automated testing
- Deployment

3. Enable CI/CD Automation
Use GitHub Actions to automate the pipeline workflow.

4. Create Workflow Directory
Inside the repository, create the directory .github/workflows/ to store workflow configuration files.

5. Create a Workflow File
Add a workflow file (e.g., ci-cd-pipeline.yml) that defines the pipeline steps and triggers.

6. Configure Workflow Trigger
Set the workflow to run automatically when code is pushed or when a pull request is created.

7. Add Continuous Integration Steps
Include steps to:

- Check out the repository code
- Install dependencies

- Build the application
- Run automated tests

8. Add Continuous Deployment Steps
Configure deployment steps to publish the application to a hosting service or server after successful builds and tests.

9. Use Environment Variables and Secrets
Store sensitive data such as API keys or credentials securely using repository secrets.

10. Commit and Push the Workflow
Save the workflow file and push the changes to the repository.

11. Monitor Pipeline Execution
Check the Actions tab in GitHub to monitor pipeline runs, view logs, and ensure all stages complete successfully.

12. Validate Deployment
After successful execution, verify that the application is deployed and functioning correctly.

Result

The full CI/CD pipeline was successfully designed and implemented using GitHub Actions in the GitHub repository.